LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Struktur Data Queue Berbasis Array dan LinkedList pada Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI i

KATA PENGANTAR ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 2

2.1 Program QueueArray 2

2.2 Program QueueArrayDriver 3

2.3 Program QueueLinkedList 4

2.4 Program ReverseData 5

2.5 Program IterasiQueue 5

BAB III TUGAS 7

3.1 Program AntrianLoket 7

BAB IV KESIMPULAN 11

4.1 Ringkasan Hasil Praktikum 11

DAFTAR PUSTAKA iii

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Implementasi Struktur Data Queue Berbasis Array dan LinkedList pada Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan struktur data, sering kali urutan kedatangan elemen menentukan urutan pemrosesannya. Konsep ini dikenal dengan prinsip First In, First Out (FIFO), di mana data yang pertama kali masuk akan menjadi data yang pertama kali keluar. Struktur data yang merepresentasikan mekanisme ini disebut dengan Queue (Antrean).

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data Queue beserta prinsip operasi utamanya yaitu First In, First Out (FIFO).
2. Mengimplementasikan operasi fundamental Queue secara manual menggunakan Array, meliputi enqueue, dequeue, front, dan rear.
3. Menerapkan antarmuka Queue menggunakan LinkedList serta manipulasi data tingkat lanjut, seperti iterasi elemen dan pembalikan urutan data (reverse) menggunakan bantuan struktur data Stack.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Mampu merancang sistem antrean data yang efisien.
2. Meningkatkan logika dalam menggabungkan dua struktur data yang berbeda (seperti Queue dan Stack) untuk menyelesaikan masalah komputasi yang lebih kompleks

BAB 2

PEMBAHASAN

2.1 Program QueueArray

```
package pekan4_2511532001;

public class QueueArray_2511532001 {
	int front_2001, rear_2001, size_2001;
	int capacity_2001;
	int array_2001[];

	public QueueArray_2511532001 (int capacity_2001) {
		this.capacity_2001 = capacity_2001;
		front_2001 = this.size_2001 = 0;
		rear_2001 = capacity_2001 - 1;
		array_2001 = new int [this.capacity_2001];
	}

	boolean isFull_2001(QueueArray_2511532001 queue_2001) {
		return (queue_2001.size_2001 == queue_2001.capacity_2001);
	}
	boolean isEmpty_2001(QueueArray_2511532001 queue_2001) {
		return (queue_2001.size_2001 == 0);
	}

	void enqueue_2001(int item_2001) {
		if (isFull_2001(this))
			return;
		this.rear_2001 = (this.rear_2001 + 1) % this.capacity_2001;
		this.array_2001 [this.rear_2001] = item_2001;
		this.size_2001 = this.size_2001 + 1;
		System.out.println(item_2001 + " enqueued to queue");
	}

	int dequeue_2001() {
		if(isEmpty_2001(this))
			return Integer.MAX_VALUE;
		int item_2001 = this.array_2001[this.front_2001];
		this.front_2001 = (this.front_2001 + 1) % this.capacity_2001;
		this.size_2001 = this.size_2001 -1;
		return item_2001;
	}
	int front_2001() {
		if (isEmpty_2001(this))
			return Integer.MIN_VALUE;
		return this.array_2001[this.front_2001];

	}
```

```
        int rear_2001() {
                if (isEmpty_2001(this))
                        return Integer.MIN_VALUE;
                return this.array_2001[this.rear_2001];
        }

        // mencetak
        void display() {
                int i;
                if (front_2001 == rear_2001) {
                        System.out.printf("\nAntrian Kosong\n");
                        return;
        }
        // kunjungi dari belakang dan cetak
        for (i = front_2001; i < rear_2001; i++) {
                System.out.printf(" %d <--", array_2001[i]);
        }
        return;

        }
}
```

Program QueueArray merupakan implementasi Queue menggunakan memori statis berupa Array. Jika elemen belakang (rear) sudah mencapai batas maksimum kapasitas array, ia dapat memutar kembali ke indeks awal (0) selama masih ada ruang kosong yang ditinggalkan oleh operasi dequeue. Program ini dilengkapi dengan validasi isFull dan isEmpty untuk mencegah error

2.2 Program QueueArrayDriver

```
package pekan4_2511532001;

public class QueueArrayDriver_2511532001 {

    public static void main(String[] args) {
        QueueArray_2511532001 queue_2001 = new QueueArray_2511532001(1000);
        queue_2001.enqueue_2001(10);
        queue_2001.enqueue_2001(20);
        queue_2001.enqueue_2001(30);
        queue_2001.enqueue_2001(40);
        System.out.println("Item di depan " + queue_2001.front_2001());
        System.out.println("Item paling belakang " +
queue_2001.rear_2001());
        System.out.println("tampilan queue");
        queue_2001.display();
        System.out.println();
        System.out.println(queue_2001.dequeue_2001() + " dihapus dari
queue");
        System.out.println("Item di depan " + queue_2001.front_2001());
        System.out.println("Item dibelakang " + queue_2001.rear_2001());
```

```
        System.out.println("tampilan queue setelah satu data dihapus");
        queue_2001.display();
    }
}
```

Program QueueArrayDriver berfungsi sebagai program utama (driver) untuk kelas QueueArray. Data dimasukkan berurutan (10, 20, 30, 40) menggunakan method enqueue. Saat fungsi dequeue dieksekusi, nilai 10 adalah data pertama yang dihapus dari struktur, membuktikan bahwa program berjalan mengikuti hukum FIFO. Pengujian fungsionalitas front, rear, dan display juga berjalan dengan menampilkan posisi elemen terdepan dan terakhir.

2.3 Program QueueLinkedList

```
package pekan4_2511532001;
import java.util.*;

public class QueueLinkedList_2511532001 {
      public static void main (String[]args) {
            Queue<Integer> q_2001 = new LinkedList<>();
            // tambah elemen
            for (int i = 0; i < 6; i++)
                  q_2001.add(i);
            // Menampilkan isi
            System.out.println("Elemen Antrian" + q_2001);
            // Untuk menghapus kepala antrian
            int hapus_2001 = q_2001.remove();
            System.out.println("Hapus Elemen" + hapus_2001);
            System.out.println(q_2001);
            // Untuk melihat antrian terdepan
            int depan_2001 = q_2001.peek();
            System.out.println("Kepala Antrian " + depan_2001);

            int banyak_2001 = q_2001.size();
            System.out.println("Size Antrian " + banyak_2001);


      }
}
```

Program QueueLinkedList memanfaatkan Java Collections Framework. Karena Queue di Java merupakan sebuah interface, instansiasinya dilakukan menggunakan kelas LinkedList. Alokasi memori menjadi dinamis tanpa perlu mendefinisikan kapasitas maksimum. Program memanggil method bawaan seperti add() untuk enqueue, remove() untuk dequeue, dan peek() untuk melihat elemen terdepan tanpa menghapusnya.

2.4 Program ReverseData

```
package pekan4_2511532001;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532001 {
	public static void main (String[]args) {
		Queue<Integer> q_2001 = new LinkedList<Integer>();
		q_2001.add(1);
		q_2001.add(2);
		q_2001.add(3);
		System.out.println("Sebelum reverse " + q_2001);
		Stack<Integer> s_2001 = new Stack<Integer>();
		while (!q_2001.isEmpty()) {
			s_2001.push(q_2001.remove());
		}
		while (!s_2001.isEmpty()) {
			q_2001.add(s_2001.pop());
	}
		System.out.println("Sesudah reverse " + q_2001);
	}
}
```

Program ReverseData menampilkan implementasi algoritma dengan menggabungkan karakteristik Queue (FIFO) dan Stack (LIFO). Tujuan utama program ini adalah membalikkan urutan data (1, 2, 3 menjadi 3, 2, 1). Logikanya dieksekusi dalam dua tahap perulangan. pertama, seluruh data antrean di-remove dan di-push ke dalam tumpukan (Stack), kedua, data di dalam Stack di-pop (diambil dari yang paling terakhir masuk) dan di add kembali ke dalam antrean. Sehingga, urutan data antrean berhasil dibalik.

2.5 Program IterasiQueue

```
package pekan4_2511532001;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Iterator;

public class IterasiQueue_2511532001 {
	public static void main (String[]args) {
		Queue<String> q_2001 = new LinkedList<>();

		q_2001.add("Praktikum");
		q_2001.add("Struktur");
		q_2001.add("Data");
```

```
            q_2001.add("Dan");
            q_2001.add("Algoritma");
            Iterator <String> iterator_2001 = q_2001.iterator();
            while (iterator_2001.hasNext()) {
                  System.out.print(iterator_2001.next() + " ");
            }
      }

}
```

Program IterasiQueue mendemonstrasikan cara menelusuri elemen-elemen di dalam antrean tanpa harus mengubah atau menghapus data aslinya. Program menggunakan Iterator bertipe String. Perulangan while dipadukan dengan method hasNext() untuk mengecek ketersediaan elemen berikutnya, dan next() untuk mencetak elemen tersebut ke layar. Penggunaan iterator membuat proses pembacaan data antrean aman dan efisien dari elemen terdepan hingga terbelakang.

BAB 3

TUGAS

3.1 Program AntrianLoket

```
package pekan4_2511532001;
import java.util.Scanner;

public class AntrianLoket_2511532001 {
    int front;
    int rear;
    int max;
    String[] queue;

    // konstruktor
    public AntrianLoket_2511532001(int kapasitas) {
        this.max = kapasitas;
        this.queue = new String[max];
        this.front = -1;
        this.rear = -1;
    }

    // isEmpty
    boolean isEmpty() {
        return front == -1;
    }

    // isFull
    boolean isFull() {
        return rear == max - 1;
    }

    // enqueue
    void enqueue(String nama) {
        if (isFull()) {
            System.out.println("Antrian penuh");
            return;
        }
        if (isEmpty()) {
            front = 0;
        }
        queue[++rear] = nama;
        System.out.println("Data berhasil ditambahkan ke antrian");
    }

    // dequeue
    void dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong");
            return;
```

```
        }
        String nama = queue[front];
        if (front == rear) { // jika hanya ada 1 data
            front = -1;
            rear = -1;
        } else {
            front++;
        }
        System.out.println(nama + " telah dilayani");
    }

    // display
    void display() {
        if (isEmpty()) {
            System.out.println("Antrian kosong");
            return;
        }
        System.out.println("Isi antrian:");
        int no = 1;
        for (int i = front; i <= rear; i++) {
            System.out.println(no + ". " + queue[i]);
            no++;
        }
    }

    // reverse
    void Reverse() {
        if (isEmpty()) {
            System.out.println("Antrian kosong");
            return;
        }
        int i = front;
        int j = rear;
        // tukar posisi elemen menggunakan perulangan
        while (i < j) {
            String temp = queue[i];
            queue[i] = queue[j];
            queue[j] = temp;
            i++;
            j--;
        }
        display(); // tampilkan langsung setelah direverse
    }

    // main class
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // inisialisasi antrian dengan kapasitas 10
        AntrianLoket_2511532001 loket = new AntrianLoket_2511532001(10);
        int pilihan;

            do {
```

```
101.                    System.out.println("\n=== PROGRAM ANTRIAN LOKET
    ===");
102.                    System.out.println("1. Tambah Antrian");
103.                    System.out.println("2. Hapus Antrian");
104.                    System.out.println("3. Tampilkan Antrian");
105.                    System.out.println("4. Reverse");
106.                    System.out.println("5. Keluar");
107.                    System.out.print("Pilih menu: ");
108.
109.                    pilihan = sc.nextInt();
110.                    sc.nextLine(); // membersihkan buffer enter
111.
112.                    switch (pilihan) {
113.                        case 1:
114.                            System.out.print("Masukkan nama pelanggan:
    ");
115.                            String nama = sc.nextLine();
116.                            loket.enqueue(nama);
117.                            break;
118.                        case 2:
119.                            loket.dequeue();
120.                            break;
121.                        case 3:
122.                            loket.display();
123.                            break;
124.                        case 4:
125.                            loket.Reverse();
126.                            break;
127.                        case 5:
128.                            System.out.println("Program selesai");
129.                            break;
130.                        default:
131.                            System.out.println("Pilihan tidak valid");
132.                    }
133.                } while (pilihan != 5);
134.                sc.close();
135.            }
136.        }
```

Program AntrianLoket dibuat mengikuti pedoman struktur data Queue. Program menggunakan Array bertipe String yang dinamakan queue[] untuk menyimpan nama-nama pelanggan. Manajemen posisi antrean dikontrol oleh dua pointer bertipe integer: front sebagai penunjuk indeks pelanggan terdepan, dan rear sebagai penunjuk indeks pelanggan paling belakang. Selain itu, terdapat variabel max yang bertugas membatasi kapasitas maksimal Array guna mencegah error OutOfBounds

Operasi penambahan data (enqueue()) bekerja dengan mengecek ketersediaan ruang lewat isFull(), kemudian menginkrementasi nilai rear dan memasukkan string nama. Sebaliknya, operasi pelayanan/penghapusan (dequeue()) mengambil elemen pada indeks front dan memajukan *pointer* front tersebut satu langkah ke belakang (inkrementasi) setelah divalidasi ketersediaan isinya.

Program ini juga dilengkapi dengan fitur membalik antrean (Reverse()) yang bekerja dengan melakukan pertukaran elemen (swapping) dari ujung front dan rear secara bersamaan menggunakan perulangan while. Seluruh logika ini dikendalikan melalui antarmuka pada method main yang menggunakan kombinasi do-while dan switch-case, sehingga pengguna dapat melakukan simulasi operasional loket secara berulang.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi struktur data Queue menggunakan Array efektif untuk mensimulasikan sistem antrean dengan prinsip pemrosesan First In, First Out (FIFO). Pengelolaan alur masuk dan keluarnya data bergantung pada manajemen dua penunjuk indeks utama, yaitu front untuk mengambil elemen terdepan yang akan dilayani (dequeue), dan rear untuk menyisipkan elemen pelanggan baru di barisan belakang (enqueue), dengan validasi isEmpty dan isFull. Memori statis Array juga memiliki fleksibilitas manipulasi yang baik, terbukti dari penerapan method Reverse yang mampu membalikkan urutan elemen antrean melalui teknik pertukaran data (swapping) secara kedua ujung indeks tanpa mengonsumsi memori tambahan.

DAFTAR PUSTAKA

[1] W3Schools, “Data Structures and Algorithms - Queues”. [Daring]. Tersedia pada: https://www.w3schools.com/dsa/dsa_data_queues.php. [Diakses: 28 April 2026]